Software Architecture and Design Specification

Project: Traffic Management System (Simulation)

Version: 1.0

Authors: Venkata Srivaibhav Nukaraju; V Vyas; Vikas N Naik; Vikhyath Bharadwaj K S

Date: 14-09-2026

Status: Final submission

Revision History

Version	Date	Author	Change summary	Approval
0.2	2026-09-07	Venkata Srivaibhav Nukaraju V Vyas Vikas N Naik Vikhyath Bharadwaj K S	Finalized simplified SRS for implementation and review.	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Traffic Management System (Simulation), a microscopic road-traffic simulator for designing and evaluating traffic-signal control strategies.

1.2 Scope

Covers interactive road-network modelling, fixed-time traffic-signal configuration, traffic-demand definition, time-stepped microscopic simulation, 2D visualization, metrics and results, run comparison, CSV export, and a local scenario library. Real-world traffic signals, sensors, field hardware, 3D visualization, distributed simulation, advanced lane-changing, and public transport/pedestrian/cyclist micro-simulation are out of scope.

1.3 Audience
Development team, Project Guide, Course Coordinator, Evaluators, future student maintainers, QA/testers, and traffic analysts performing acceptance testing.

1.4 Definitions
TMS-Sim: Traffic Management System (Simulation); FR/NFR: Functional/Non-Functional Requirement; RTM: Requirements Traceability Matrix; MOE: Measure of Effectiveness; PRNG/Seed: Pseudo-Random Number Generator and reproducibility seed; FPS: Frames Per Second; Real-time factor: simulated seconds advanced per wall-clock second; Car-following model and Gap acceptance: documented vehicle movement rules.

2. Document Overview

2.1 How to use this document
Provides the architectural deliverables for TMS-Sim, including architecture rationale, component descriptions, UML diagram placeholders, design overview, internal interface/API definitions, security architecture, error handling, UX considerations, risks, traceability, and future work.

2.2 Related Documents
Software Requirements Specification (SRS) for Traffic Management System (Simulation), Requirements Traceability Matrix (RTM), course material for Software Engineering UE24CS341A, and the documented simulation model/reference material.

3. Architecture

3.1 Goals & Constraints
Goals: provide a clear desktop workbench for traffic analysts; support reproducible microscopic simulation; validate road networks and signal plans; visualize traffic behaviour in 2D; calculate comparable traffic measures; and support maintainable, testable modules.
Constraints: Windows 10/11 desktop environment, Python 3.11 + PySide6, local SQLite database or scenario files, single-machine execution, 2D visualization, documented car-following and gap-acceptance rules, and no required network communication.

3.2 Stakeholders & Concerns
Traffic Analyst: understandable workflow, clear validation, simulation control, visualization, metrics and comparison. Development Team: modularity, maintainability and testability. Project Guide/Course Coordinator/Evaluators: correctness, traceability and acceptance evidence. QA/Testers: reproducibility and measurable acceptance criteria. Future Student Maintainers: understandable structure and separable modules.

3.3 Component (UML) Diagram

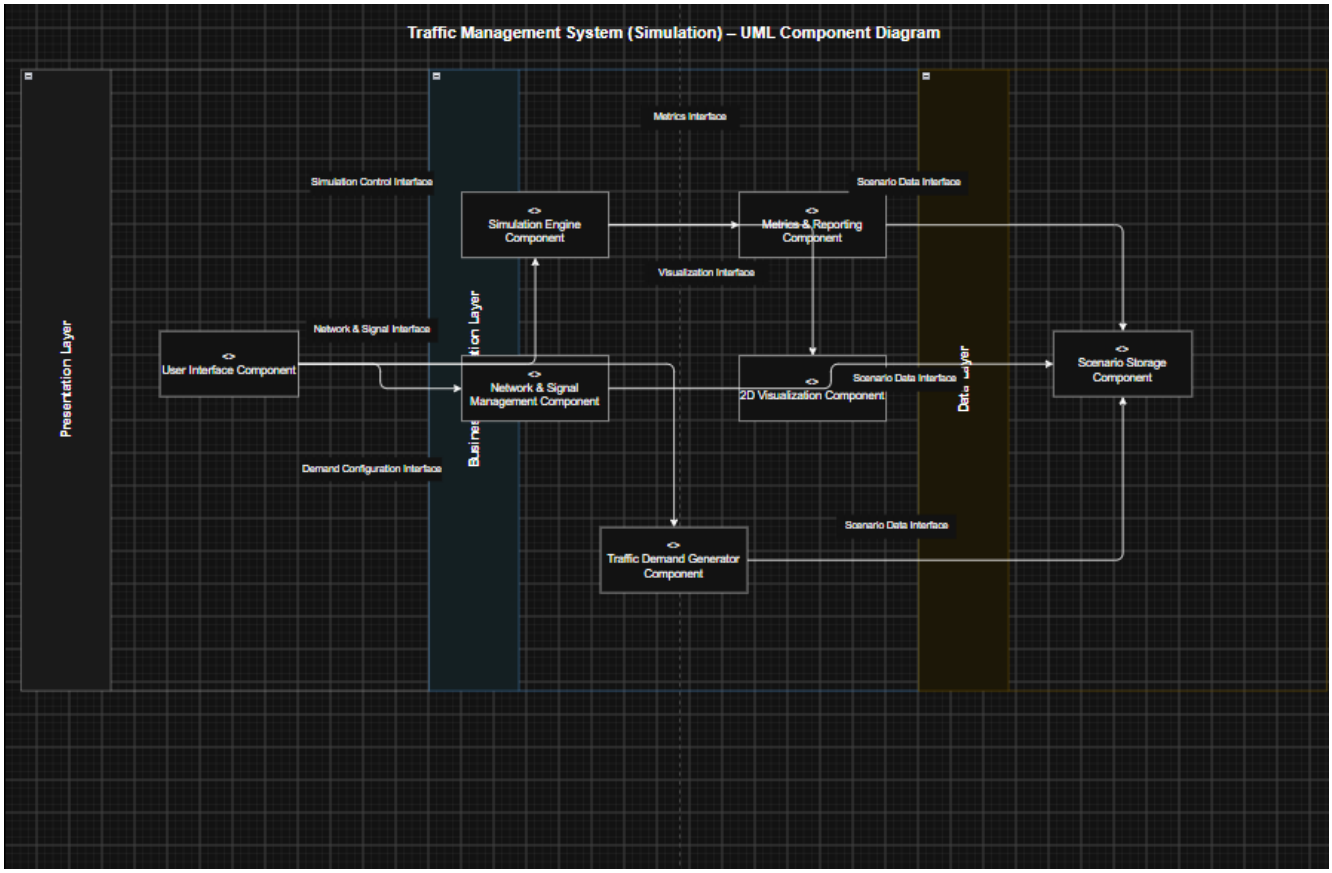


Figure 1: UML Component Diagram – Traffic Management System (Simulation)

3.4 Component Descriptions

- Network Editor: creates and edits intersections, road segments, lanes and turning movements and validates the network.
- Signal Configuration: defines fixed-time phases and validates timing and movement conflicts.
- Demand Generator: defines entry-point arrival rates and turning ratios and generates reproducible vehicle arrivals using a seedable PRNG.
- Simulation Engine: advances the simulation in fixed time steps using the documented car-following and gap-acceptance rules and provides simulation controls.
- 2D Renderer: displays vehicle positions, signal indications and queues with pan and zoom.
- Metrics Service: computes delay, queue length, throughput, average speed and travel time and supports result comparison.
- Scenario Store: saves, lists, loads, renames and deletes scenarios and completed runs, using local storage.
- Report Exporter: exports completed-run summary metrics as CSV.

3.5 Chosen Architecture Pattern and Rationale

Layered architecture is selected for TMS-Sim to separate presentation, simulation/business logic, and data/reporting responsibilities. This supports the SRS quality attributes of maintainability and testability because simulation, UI, storage and metrics can be separated into modules. A microservices/distributed approach is not appropriate for the current scope because the SRS specifies a software-only desktop application with single-machine execution and no required network communication.

3.6 Technology Stack & Data Stores

Python 3.11 with PySide6 for the desktop application. Local storage uses SQLite or a scenarios folder on disk. Scenario files use version-tagged JSON, and completed-run summary metrics can be exported as CSV. The application does not require an internet connection for modelling or simulation.

3.7 Risks & Mitigations

Risk: invalid or disconnected road networks or invalid signal timings could produce incorrect runs → Mitigation: validate the network and signal plan before execution and block unsafe/invalid runs. Risk: non-reproducible traffic demand → Mitigation: use a single seedable PRNG. Risk: visualization performance degradation on larger scenarios → Mitigation: measure against the reference hardware and maintain the SRS target of at least 15 FPS for the reference scenario.

3.8 Traceability to Requirements

TMS-F-010…TMS-F-013 → Network Editor; TMS-F-020…TMS-F-022 → Signal Configuration; TMS-F-030…TMS-F-032 → Demand Generator; TMS-F-040…TMS-F-043 → Simulation Engine / 2D Renderer; TMS-F-050…TMS-F-054 → Metrics Service / Report Exporter; TMS-F-060 → Scenario Store; TMS-NF-001 → Performance; TMS-NF-002…TMS-NF-004 → Reliability, Usability and Portability.

3.9 Security Architecture

The SRS includes user management and access control and defines security as a non-functional area. The architecture should enforce access control at the application boundary before protected scenario, configuration and result operations are performed. Since TMS-Sim is a local desktop application and requires no network communication, the design focuses on controlled application access and protection of local scenario/run data. Detailed security objectives and requirements remain governed by SRS §5.1.

4. Design

4.1 Design Overview

TMS-Sim is designed as a modular desktop system. The presentation/workbench layer provides network editing, signal configuration, demand input, simulation controls and result visualization. Core simulation services validate inputs, generate vehicles, execute the

time-stepped model and calculate metrics. Local storage manages scenarios and completed runs, while reporting exports summary metrics for analysis.

4.2 UML Sequence Diagrams

flows : (1) Run Simulation — Analyst → Workbench → Network/Signal Validation → Demand Generator → Simulation Engine → Metrics Service → Scenario Store/Results; (2) Compare and Export Results — Analyst → Results Dashboard → Metrics Service → Scenario Store → Report Exporter.

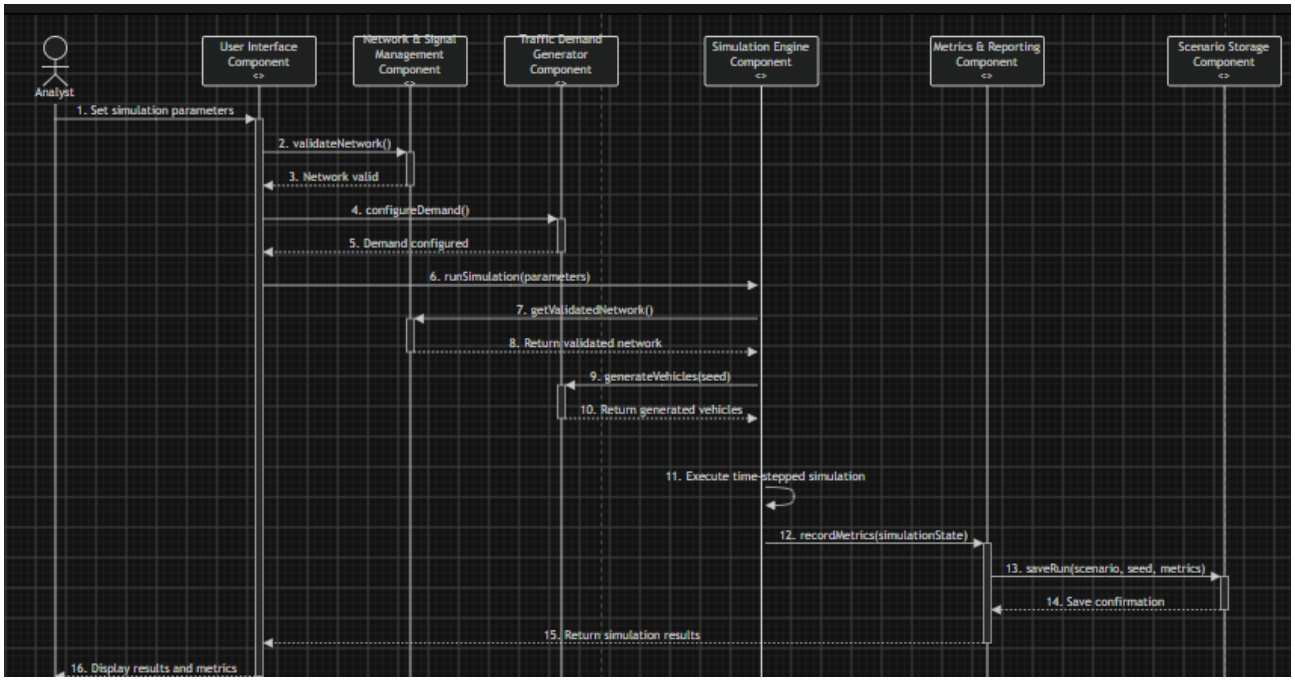


Figure 2: UML Sequence Diagram – Run Simulation

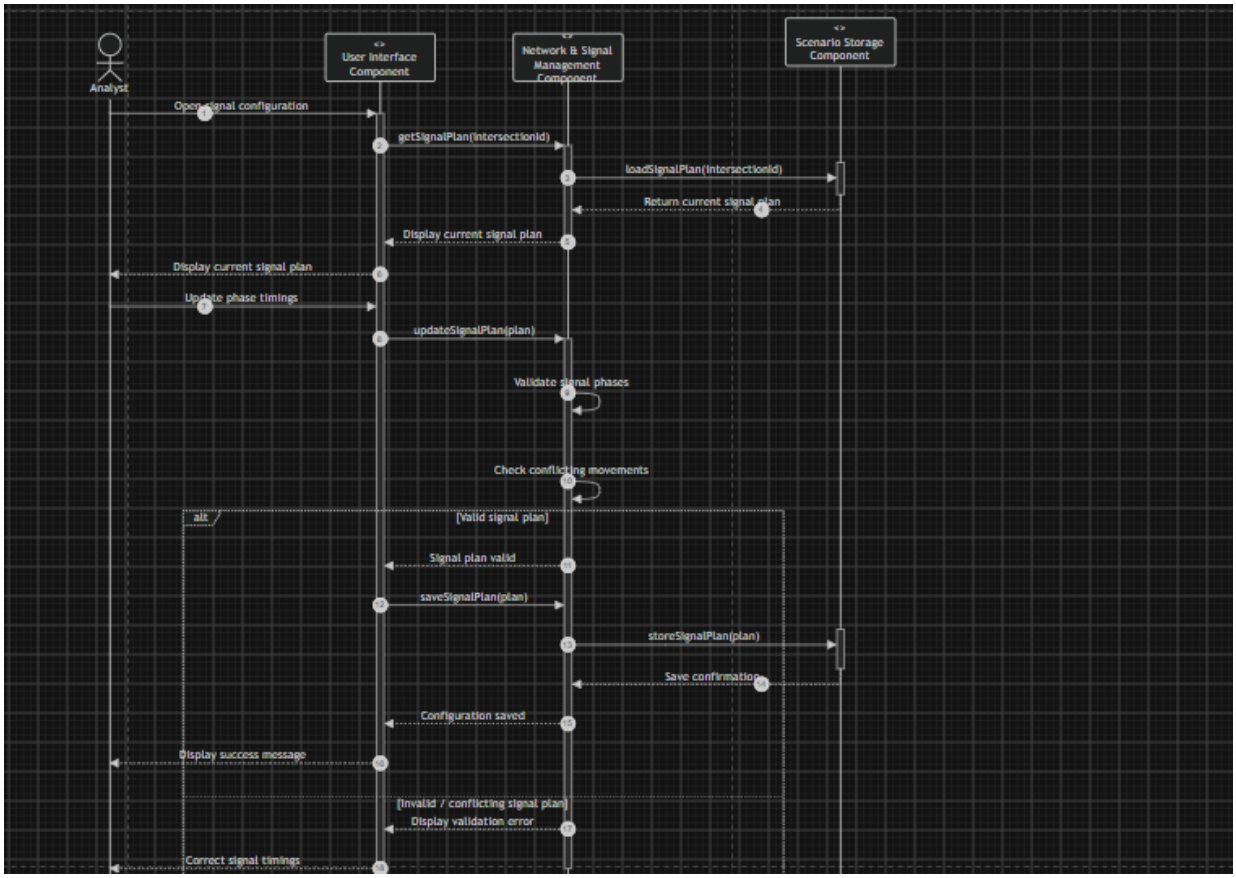


Figure 3: UML Sequence Diagram – Configure Traffic Signal Plan

4.3 API Design

TMS-Sim is a local desktop application, so no external REST API is required. The following logical component interfaces should be defined and documented in the implementation.

Interface 1: Network Editor → Simulation Engine
Operations: validateNetwork(), loadNetwork(), provideNetworkState().
Interface 2: Signal Configuration → Simulation Engine
Operations: validateSignalPlan(), provideSignalPlan().
Interface 3: Demand Generator → Simulation Engine
Operations: generateVehicles(seed, arrivalRates, turningRatios), provideDemand().
Interface 4: Simulation Engine → Metrics Service
Operations: recordVehicleEvents(), computeMetrics().
Interface 5: Scenario Store ↔ Workbench
Operations: saveScenario(), loadScenario(), saveRun(), listScenarios().

4.4 Error Handling, Logging & Monitoring

Validation errors should be presented clearly and linked to the offending network or configuration element where possible. The system should block a run when the SRS-defined network or signal safety validation fails. Simulation and result errors should use standardized messages. Monitoring/acceptance evidence should include simulation completion, visualization performance, reproducibility, and key metric correctness.

4.5 UX Design

The workbench uses a single window with a central network canvas, properties/inspector panel, palette/toolbox, simulation control bar, and dockable results dashboard. Direct manipulation supports placing intersections, drawing segments and editing properties. Validation errors are shown in a list, and the results dashboard provides live charts during a run and summary tables/charts after completion.

4.6 Open Issues & Next Steps

Complete the UML component diagram and at least two sequence diagrams during detailed design. Finalize component-level interfaces and implementation responsibilities. Continue implementation and acceptance testing against the RTM. Future scope may consider capabilities outside the current v1 constraints only after the core requirements are satisfied.

5. Appendices

5.1 Glossary: TMS-Sim, SRS, FR, NFR, RTM, MOE, PRNG, Seed, FPS.

5.2 References: IEEE Std 830-1998; course material “Requirements Engineering: Software Requirement Specification, RTM”, Unit 1, Lecture 9–10 (UE24CS341A); course SRS template; Treiber, Hennecke, Helbing (2000), “Congested traffic states in empirical observations and microscopic simulations”.

5.3 Tools: Python 3.11, PySide6, SQLite/JSON local storage, CSV export, draw.io or PlantUML for UML diagrams.